

# CIS 5357

## Computing for Data Analytics

Week 6

22 FEBRUARY 2023

# Last week

- functions
- common packages (libraries)
- web scraping
- coding assignment

# Quick refresher

- Functions are snippets of code that you can reuse over and over
- What keyword do you use to create a function? `def function(input1, input2, ...):`
- A library is a generic term used to describe a bunch of code that was designed to be used by many different scripts or applications
- The Python Standard Library is a set of built-in packages that comes with Python
- 3rd party packages from other coders can also be used in Python code and loaded using the command line command pip
- Web scraping is the process of extracting data from websites

Module 5

Coding assignment

### Instructions

Imagine you are a marketing analyst, and your boss has asked you to analyze the effectiveness of marketing campaigns. They want you to evaluate each campaign's reach, engagement rate, and conversion rate to recommend whether to continue, modify, or discontinue a campaign based on its performance. Use Python and the requirements below to fulfill this request.

- Continue with any marketing campaign that has engagement rate above 15% or conversion rate above 5%.
- Any campaign with a conversion rate below 1% or reach below 3,000 should be discontinued, regardless of the engagement rate.
- Any campaigns not meeting the above criteria should be flagged for review and possible modification.

Below is a small sample of the marketing campaign performance data available to test your code.

| Name               | Reach   | Engagement | Conversion |
|--------------------|---------|------------|------------|
| Recommendations    | 15,234  | 1,451      | 523        |
| Product Highlights | 2,030   | 750        | 35         |
| Monthly Newsletter | 25,031  | 1,120      | 2,123      |
| Social Contest     | 230,981 | 1,250      | 1,124      |

- Step 1: load the data
- Step 2: calculate campaign performance
- Step 3: determine results
- Step 4: return results

# One solution

```
# 1. store test data in lists
campaign = ['Recommendations', 'Product Highlights', 'Monthly Newsletter', 'Social Contest']
reach = [15234, 2030, 25031, 230981]
engagement = [1451, 750, 1120, 1250]
conversion = [523, 35, 2123, 1124]
```

# One solution

```
# 1. store test data in lists
campaign = ['Recommendations', 'Product Highlights', 'Monthly Newsletter', 'Social Contest']
reach = [15234, 2030, 25031, 230981]
engagement = [1451, 750, 1120, 1250]
conversion = [523, 35, 2123, 1124]

# 2. calculate engagement rate and conversion rate
engagement_rate, conversion_rate = [], []
for i in range(len(campaign)):
    engagement_rate.append(engagement[i]/reach[i])
    conversion_rate.append(conversion[i]/reach[i])
```

# One solution

```
# 1. store test data in lists
campaign = ['Recommendations', 'Product Highlights', 'Monthly Newsletter', 'Social Contest']
reach = [15234, 2030, 25031, 230981]
engagement = [1451, 750, 1120, 1250]
conversion = [523, 35, 2123, 1124]

# 2. calculate engagement rate and conversion rate
engagement_rate, conversion_rate = [], []
for i in range(len(campaign)):
    engagement_rate.append(engagement[i]/reach[i])
    conversion_rate.append(conversion[i]/reach[i])

# 3. define the decision logic
results = []
for i in range(len(campaign)):
    if conversion_rate[i] < 0.01 or reach[i] < 3000:
        results.append('DISCONTINUE')
    elif engagement_rate[i] > 0.15 or conversion_rate[i] > 0.05:
        results.append('CONTINUE')
    else:
        results.append('REVIEW')
```

# One solution

```
# 1. store test data in lists
campaign = ['Recommendations', 'Product Highlights', 'Monthly Newsletter', 'Social Contest']
reach = [15234, 2030, 25031, 230981]
engagement = [1451, 750, 1120, 1250]
conversion = [523, 35, 2123, 1124]

# 2. calculate engagement rate and conversion rate
engagement_rate, conversion_rate = [], []
for i in range(len(campaign)):
    engagement_rate.append(engagement[i]/reach[i])
    conversion_rate.append(conversion[i]/reach[i])

# 3. define the decision logic
results = []
for i in range(len(campaign)):
    if conversion_rate[i] < 0.01 or reach[i] < 3000:
        results.append('DISCONTINUE')
    elif engagement_rate[i] > 0.15 or conversion_rate[i] > 0.05:
        results.append('CONTINUE')
    else:
        results.append('REVIEW')

# 4. display the results
for i in range(len(campaign)):
    print(f"{campaign[i]}: Reach {reach[i]:,} - Engagement Rate {engagement_rate[i]*100:.2f}% - Conversion  
Rate {conversion_rate[i]*100:.2f}% - {results[i]}")
```

# Test data results

**Recommendations:** Reach 15,234 - Engagement Rate 9.52% - Conversion Rate 3.43% - **REVIEW**

**Product Highlights:** Reach 2,030 - Engagement Rate 36.95% - Conversion Rate 1.72% - **DISCONTINUE** (low reach)

**Monthly Newsletter:** Reach 25,031 - Engagement Rate 4.47% - Conversion Rate 8.48% - **CONTINUE**

**Social Contest:** Reach 230,981 - Engagement Rate 0.54% - Conversion Rate 0.49% - **DISCONTINUE** (low engagement)

# What if...

- thousands of campaigns to analyze?
- asked to do this regularly?
- your boss is busy and prefers summarized information?
- your boss is very detail-oriented and likes to get into specifics?

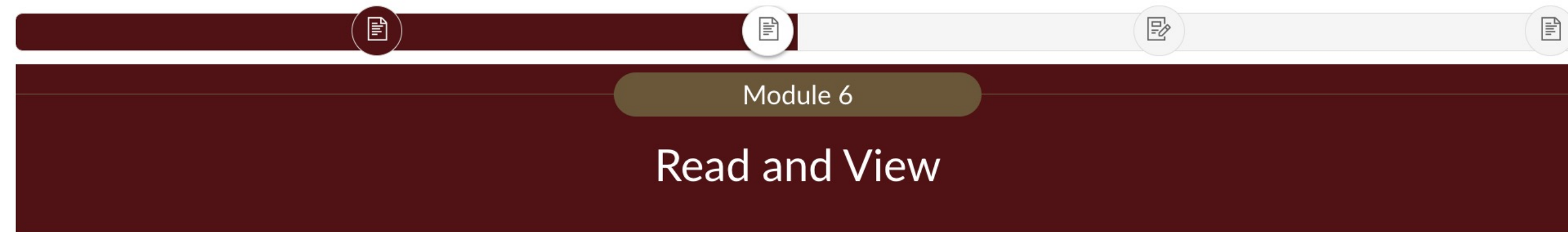
# What if...

- thousands of campaigns to analyze?
  - Use more efficient data structures like file upload; add more checks for data anomalies
- asked to do this regularly?
  - automate the process with a script that can be scheduled to run periodically, create a function, ensure that the code is robust to handle any changes in data format or logic (e.g., parameterize performance cutoffs)
- your boss is busy and prefers summarized information?
  - Add a summary report feature to the code that aggregates the decisions into a concise format, highlighting key statistics and recommendations, visualize the information
- your boss is very detail-oriented and likes to get into specifics
  - Ensure that the code provides detailed metrics for each campaign and potentially include additional analysis or visualizations to support the decision-making process

# This week

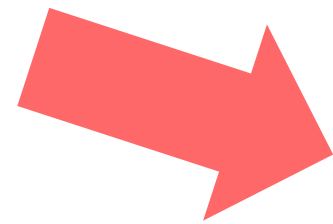
- Dictionaries
- Part 1 wrap up
- Midterm overview

# Download code to follow along



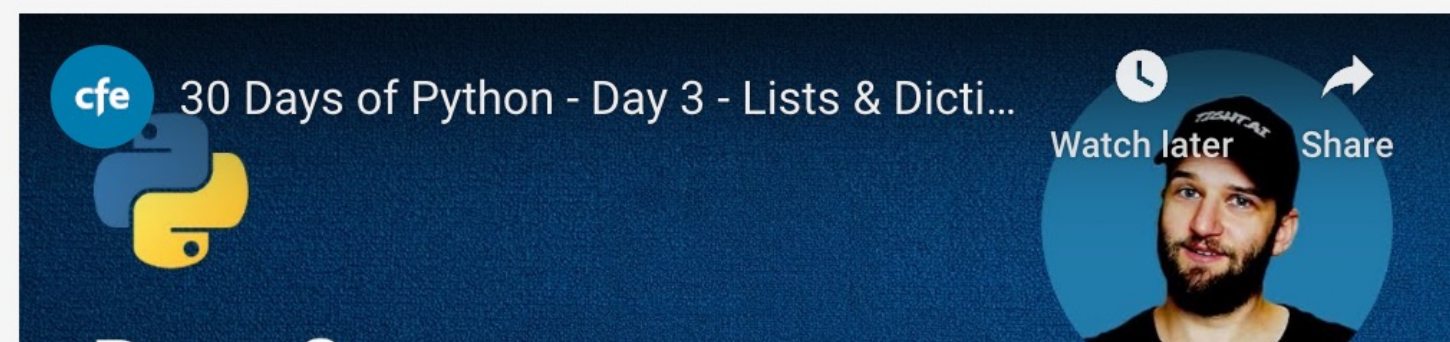
## Read

- Textbook chapter 3 pages 107-115
- [Difference between lists and dictionaries](#) ↗
- [Python documentation: 5.5 Dictionaries](#) ↗
- [Practice code from lecture](#) ↓ —follow along in class
- Lecture slides (coming soon!)



## View

[List and dictionaries](#) ↗



# Dictionaries

A Python dictionary is an **unordered collection** that stores data as key value pairs, differing from other data types that store single values.

In a dictionary, each key value pair is **delimited by a colon (:)**, and individual keys are **distinguished by commas**.

# Dictionary example

Dictionaries can be used to store information about things like users.

```
user = {'name': 'Mattan',  
        'height': 70,  
        'shoe size': 10.5,  
        'hair': 'Brown',  
        'eyes': 'Brown'}
```

# Dictionary example

Dictionaries can be used to store information about things like users.

```
user = {'name': 'Mattan',  
        'height': 70,  
        'shoe size': 10.5,  
        'hair': 'Brown',  
        'eyes': 'Brown'}
```



Dictionaries start and  
end with curly brackets

# Dictionary example

Dictionaries can be used to store information about things like users.

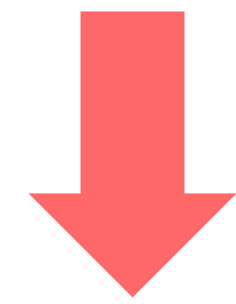


the *key* is to the left of the colon; each key is unique (meaning we can't use the same key more than once)

```
user = {'name': 'Mattan',  
        'height': 70,  
        'shoe size': 10.5,  
        'hair': 'Brown',  
        'eyes': 'Brown'}
```

# Dictionary example

Dictionaries can be used to store information about things like users.



the *value* is to the right of the colon & can be basically anything—a string, a number, a list, or even another dictionary

```
user = {'name': 'Mattan',  
        'height': 70,  
        'shoe size': 10.5,  
        'hair': 'Brown',  
        'eyes': 'Brown'}
```

# List v. Dictionaries

The difference between list and dictionary in Python lies in their structure:

- a list is an **ordered collection** indexed by position and allowing duplicates
- a dictionary is an unordered collection of **unique key pairs**

Lists are best for sequential data, while dictionaries are tailored for data retrieval by specific identifiers.

*Let's chat*

What could you use a  
Python dictionary for?

# Potential business applications

- **Managing user inputs:** store user-entered data for further processing and usage within the program
- **Frequency tables:** create frequency tables where keys are the objects for which frequency needs to be counted, and the values are the frequencies; useful for analyzing and summarizing large datasets
- **Configuration data storage:** store configuration data like database connection parameters

# Create a dictionary

Creating a dictionary is as simple as placing items inside curly braces **{}** separated by a comma or by using **dict()**

The bracket method

```
data = {  
    "beer_data": beers,  
    "brewery_data": breweries  
}
```

Key Value

The dict() method, option 1

```
data2 = dict(  
    beer_data=beers,  
    brewery_data=breweries  
)
```

Key Value

Each element in a dictionary is represented by **key:value** pair

The values can be any data type and can repeat, **keys must be unique**

# Another dictionary example

A Python dictionary is a fast, versatile way to store and retrieve data by way of a name, rather than just an index number.

```
countryCapitals = {'Spain': 'Madrid',  
                  'France': 'Paris',  
                  'Germany': 'Berlin',  
                  'Norway': 'Oslo'}
```

# Access items in a dictionary

Strings and lists use indexing to access values but **dictionaries use keys** to access items.

Key can be used either inside square brackets `[]` or with the `get()` method.

```
print(countryCapitals['France']) #uses square brackets
```

```
print(countryCapitals.get('France')) #no square brackets; safer approach
```

# Add/change elements in a dictionary

Dictionaries are **mutable**—we can add new items or change the value of existing items using the assignment operator.

If the **key already exists the value get updated**, otherwise a new *key:value* pair is added to the dictionary.

```
# Add India to the list
countryCapitals['India'] = 'dehli'
print(countryCapitals)
```

```
# Oops, let's capitalize that
countryCapitals['India'] = 'Dehli' # updates the value of the India key
print(countryCapitals)
```

# Remove elements in a dictionary

We can remove a particular item in a dictionary using the method ***pop()***. This removes an item with the provide key and returns the value.

```
countryCapitals.pop('Norway') #removes Norway from dictionary
```

Or all the items can be removed at once using the ***clear()*** method.

```
countryCapitals.clear() # removes everything from the dictionary
```

We can also use the ***del*** keyword to remove individual items or the entire dictionary itself.

```
del countryCapitals['Norway'] #use square brackets to delete one item  
del countryCapitals #no key removes the variable all together
```

# Iterate through a dictionary

We can use a *for* loop to iterate through each key in a dictionary

```
for c in countryCapitals:  
    print(f"The capital of {c} is {countryCapitals[c]}.")
```

# Search a dictionary

We can use logic statements to search a dictionary

```
key = input("Enter a country:")  
if key in countryCapitals:  
    print(f"The capital of {key} is {countryCapitals[key]}.")  
else:  
    print(f"{key} is not present in the dictionary.")
```

# Other dictionary functions & methods

| Function | Description                                         |
|----------|-----------------------------------------------------|
| len()    | Returns the number of items in the dictionary       |
| sorted() | Returns a new sorted list of keys in the dictionary |
| str()    | Returns a string representation of the dictionary   |

| Method    | Description                                |
|-----------|--------------------------------------------|
| .copy()   | Returns a copy of the dictionary           |
| .items()  | Returns the dictionary's items (key:value) |
| .keys()   | Returns the dictionary's keys              |
| .values() | Returns the dictionary's values            |

# More loop examples

```
# print the values
print("The capital cities are:")
for v in countryCapitals.values():
    print(v)
```

# More loop examples

```
# print the values
print("The capital cities are:")
for v in countryCapitals.values():
    print(v)
```

```
# print keys and values
for k in countryCapitals.keys():
    print(f"The capital of {k} is {countryCapitals[k]}")
```

# More loop examples

```
# print the values
print("The capital cities are:")
for v in countryCapitals.values():
    print(v)
```

```
# print keys and values
for k in countryCapitals.keys():
    print(f"The capital of {k} is {countryCapitals[k]}")
```

```
for k, v in countryCapitals.items():
    print(f"The capital of {k} is {v}")
```

# Recap list v. dictionaries

| List                                                                         | Dictionary                                              |
|------------------------------------------------------------------------------|---------------------------------------------------------|
| <b>Ordered collection</b> indexed by position and <b>allowing duplicates</b> | <b>Unordered</b> collection of <b>unique key pairs</b>  |
| Best for <b>sequential</b> data                                              | Useful for data <b>retrieval</b> by specific identifier |
| Indexed by <b>range of numbers</b>                                           | Indexed by <b>unique keys</b>                           |
| Defined by <code>[]</code>                                                   | Defined by <code>{}</code>                              |

# Questions on these

Difference between lists and dictionaries

Creating a dictionary

Accessing items in a dictionary

Adding or changing items in a dictionary

Removing elements from a dictionary

Iterating through a dictionary

# Challenge 1: create a dictionary

Convert a list to a dictionary of squared values

For example [1, 2, 3] -> {1:1, 2:4, 3:9}

# Challenge 1: create a dictionary

Convert a list to a dictionary of squared values

For example [1, 2, 3] -> {1:1, 2:4, 3:9}

We'll resume in  
**8** minutes...

# Challenge 1: create a dictionary

Convert a list to a dictionary of squared values

For example [1, 2, 3] -> {1:1, 2:4, 3:9}

We'll resume in  
**7** minutes...

# Challenge 1: create a dictionary

Convert a list to a dictionary of squared values

For example [1, 2, 3] -> {1:1, 2:4, 3:9}

We'll resume in  
**6** minutes...

# Challenge 1: create a dictionary

Convert a list to a dictionary of squared values

For example [1, 2, 3] -> {1:1, 2:4, 3:9}

We'll resume in  
**5** minutes...

# Challenge 1: create a dictionary

Convert a list to a dictionary of squared values

For example [1, 2, 3] -> {1:1, 2:4, 3:9}

We'll resume in  
**4** minutes...

# Challenge 1: create a dictionary

Convert a list to a dictionary of squared values

For example [1, 2, 3] -> {1:1, 2:4, 3:9}

We'll resume in  
**3** minutes...

# Challenge 1: create a dictionary

Convert a list to a dictionary of squared values

For example [1, 2, 3] -> {1:1, 2:4, 3:9}

We'll resume in  
**2** minutes...

# Challenge 1: create a dictionary

Convert a list to a dictionary of squared values

For example [1, 2, 3] -> {1:1, 2:4, 3:9}

We'll resume in  
**1** minute...

# A few possible solutions

```
1  list = [1, 2, 3]
2
3  squares1 = {}
4  for i in list:
5      | squares1[i] = i*i
6  print(squares1)
7
```

# A few possible solutions

```
1  list = [1, 2, 3]
2
3  squares1 = {}
4  for i in list:
5      squares1[i] = i*i
6  print(squares1)
7
8  squares2 = {}
9  for i in range(len(list)):
10     squares2[i+1] = (i+1)**2
11  print(squares2)
12
```

# A few possible solutions

```
1  list = [1, 2, 3]
2
3  squares1 = {}
4  for i in list:
5      squares1[i] = i*i
6  print(squares1)
7
8  squares2 = {}
9  for i in range(len(list)):
10     squares2[i+1] = (i+1)**2
11  print(squares2)
12
13  squares3 = {x: x**2 for x in list}
14  print(squares3)
```

# Make it a function

```
def square_dict(list):  
    squares = {num: num**2 for num in list}  
    return squares
```

# Exception for non-numeric input

```
def square_dict(list):  
    try:  
        squares = {num: num**2 for num in list}  
        return squares  
    except TypeError:  
        print('Please enter a numeric list')
```

# Challenge 2: swap keys & values

Reverse keys and values in a dictionary.

For example {1:1, 2:4, 3:9} -> {1:1, 4:2, 9:3}

# Challenge 2: swap keys & values

Reverse keys and values in a dictionary.

For example {1:1, 2:4, 3:9} -> {1:1, 4:2, 9:3}

We'll resume in  
**8** minutes...

# Challenge 2: swap keys & values

Reverse keys and values in a dictionary.

For example {1:1, 2:4, 3:9} -> {1:1, 4:2, 9:3}

We'll resume in  
**7** minutes...

# Challenge 2: swap keys & values

Reverse keys and values in a dictionary.

For example {1:1, 2:4, 3:9} -> {1:1, 4:2, 9:3}

We'll resume in  
**6** minutes...

# Challenge 2: swap keys & values

Reverse keys and values in a dictionary.

For example {1:1, 2:4, 3:9} -> {1:1, 4:2, 9:3}

We'll resume in  
**5** minutes...

# Challenge 2: swap keys & values

Reverse keys and values in a dictionary.

For example {1:1, 2:4, 3:9} -> {1:1, 4:2, 9:3}

We'll resume in  
**4** minutes...

# Challenge 2: swap keys & values

Reverse keys and values in a dictionary.

For example {1:1, 2:4, 3:9} -> {1:1, 4:2, 9:3}

We'll resume in  
**3** minutes...

# Challenge 2: swap keys & values

Reverse keys and values in a dictionary.

For example {1:1, 2:4, 3:9} -> {1:1, 4:2, 9:3}

We'll resume in  
**2** minutes...

# Challenge 2: swap keys & values

Reverse keys and values in a dictionary.

For example {1:1, 2:4, 3:9} -> {1:1, 4:2, 9:3}

We'll resume in  
**1** minute...

# Some possible solutions

```
dict = {1: 1, 2: 4, 3: 9}
swap = {}
for key, value in dict.items():
    swap[value] = key
print(swap)
```

# Some possible solutions

```
dict = {1: 1, 2: 4, 3: 9}
swap = {}
for key, value in dict.items():
    swap[value] = key
print(swap)
```

```
dict = {1: 1, 2: 4, 3: 9}
swap2 = {value: key for key, value in dict.items()}
print(swap2)
```

# Make it a function

```
def reverse_dict(dict):  
    reversed_dict = {value: key for key, value in dict.items()}  
    return reversed_dict
```

# Exception for non-dictionary input

```
def reverse_dict(dict):  
    try:  
        reversed_dict = {value: key for key, value in dict.items()}  
        return reversed_dict  
    except AttributeError:  
        print('Please enter a dictionary')
```

# Attribute vs. Type Error

- **AttributeError** is raised when an attribute reference or assignment fails, such as when trying to access a non-existent attribute or method on an object.
- **TypeError** is raised when an operation or function is applied to an object of inappropriate type. For example, passing arguments of the wrong type (e.g., passing a list when an int is expected).

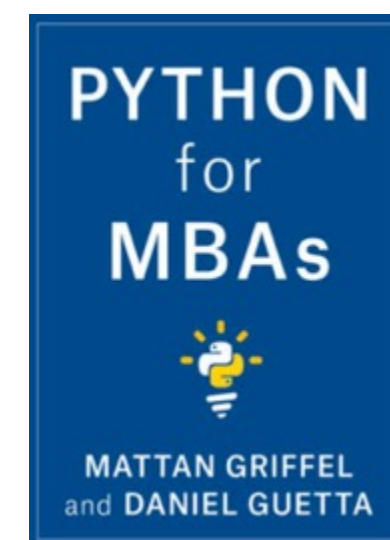
# Part 1 Wrap Up

## **PART 1: PYTHON BASICS**

Chapters 1-4

## **PART 2: DATA ANALYSIS**

Chapters 5-9



# Midterm logistics

- Virtually on Zoom **Thursday, February 29** at **6:30p**
- Up to **3 hours** to complete **5 questions**
- You **can use** your notes, lecture code, and the text
- You **cannot** contact a person to get an answer
- **Partial credit** will be given, so try not to leave any questions blank

# Midterm topics

- Debug common coding errors
- Leverage appropriate data types (integer, float, string, list, dictionary)
- Take input from a user
- Print output
- Use if/elif/else logic
- Iterate with loops
- Create and use functions
- Import packages/libraries
- Scrape web data

Class challenges and assignments are good practice for the midterm

# Quick recap

- A Python **dictionary** is an **unordered collection** that stores data as key value pairs, differing from other data types that store single values.
- A list is an **ordered collection** indexed by position and allowing duplicates, while a dictionary is an unordered collection of **unique key pairs**. Lists are best for sequential data, while dictionaries are tailored for data retrieval by specific identifiers.
- In a dictionary, each key value pair is **delimited by a colon (:)**, and individual keys are **distinguished by commas**.
- Creating a dictionary is as simple as placing items inside curly braces **{}** **separated by a comma** or by using **dict()**. The values can be any data type and can repeat, **keys must be unique**.
- Dictionaries are **mutable**—we can add new items or change the value of existing items using the assignment operator. If the key already exists the value gets updated, otherwise a new *key:value* pair is added to the dictionary.
- Similar to lists, we can also **remove elements** using methods like `.pop()` and `.clear()`.
- We can also iterate through elements using **loops** and use **logic** statements to search a dictionary.

# Next week

Review previous class challenges and assignments for practice

Meet on Zoom for the **MIDTERM**